

How I Completed Exercise 01 Using a Public Linux Machine on Campus

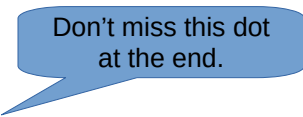
I went to room 2108 in EB3. This is a public lab with a mix of Windows and Linux machines. It looks like the Linux ones are all on one row at the back edge of the room (near the outside windows).

I logged in, filled out some first-time-setup dialogs and then opened a terminal window using the applications menu at the top, Applications → System Tools → Terminal.

The following shows some of the shell commands I used to complete the exercise. The *shell* is the command interpreter running in your terminal. The \$ at the start of each line below means I'm showing you shell commands. Don't really type the \$ at the front of each command.

In the terminal, I made a fresh directory for working on CSC 230 assignments, with a directory under that for working on exercises and one more directory under that for working on exercise 1. After changing to my new directory for exercise 1, I used the command given in the assignment to copy over the starter files from the shared CSC 230 file space.

```
$ mkdir csc230
$ cd csc230
$ mkdir ex
$ cd ex
$ mkdir ex01
$ cd ex01
$ cp /mnt/coe/workspace/csc/CSC230/ex/exercise01/* .
```



Don't miss this dot
at the end.

Then, I started an editor to work on the arithmetic.c source file. The & at the end of the command says to run the editor in background, so you still get to type more commands at your shell prompt without closing the editor. The gedit program is nice because it's a little graphical and it knows about the syntax of lots of languages, so it can do syntax highlighting for you. There are other editors you could use, including ones that run right in the terminal.

```
$ gedit arithmetic.c &
```

In the editor, I added the code I needed to and then saved the file with the big save button at the top. Then, back in my terminal window, I compiled and ran the program (I didn't close my editor window yet, in case I needed to fix anything).

The commands below compile the program with our usual compiler options. Then, I run the program, using the **tee** program to send its output to the screen and to a file named output.txt. This makes it easy to see the output right away and then to check it later to make sure it's exactly right. Then, I check the exit status of my program to make sure it exited successfully. Finally, I use the **diff** program to make sure my program's output (saved to the file named output.txt) looks exactly like the expected output.

```
$ gcc -Wall -std=c99 arithmetic.c -o arithmetic
$ ./arithmetic | tee output.txt
500000000500000000
$ echo $?
0
```

```
$ diff output.txt expected.txt
```

You should plan to check your output against the expected output on every exercises. That's what the grading program will do when we're evaluating your work. The automatic grader isn't smart enough to tell the difference between important and unimportant differences in your program's output, so be sure to use diff to make sure your output is exactly right. Don't depend on just looking at your programs output in the terminal window. This could make it easy to miss a small difference that will cost you points when the automated grader looks at your output.

Everything looks good in the sample execution above. The diff program didn't find any differences between my program's output and the expected output.

After I was happy with my solution, I brought up a web browser and navigated to the Moodle page for my section of CSC 230. I clicked on the assignment link for exercise 1, added my completed assignment.c file to my submission for the assignment and submitted it. I should be able to use this same link later to check my grade on the assignment and check on feedback for my submission.

When I was done, I closed all my windows and logged out. You're not required to complete the exercise this way (there are lots of other options), but it was pretty easy like this.